

HW04 – Automation Testing Main Report

1. Student and assignment information

- Student ID: 23127414
- Assignment: HW04 – Automation Testing
- SUT: EShop
- Automation framework: Playwright 1.62.1
- Browsers: Chromium, Firefox, WebKit
- AI declaration: I use AI tools for the following tasks: requirement analysis, test-data drafting, Playwright scaffolding, code review, execution analysis, report preparation, and Agent Skill creation. All generated work was reviewed and executed against the real SUT.

2. Feature selection

The same three web features selected in HW02 were automated:

Pool	ID	Feature
A	FR-01	Account registration
B	FR-11	User order history view
C	FR-14	Category management

The HW02 mobile feature was excluded because HW04 requires web frontend automation.

3. Automation design

The suite uses a data-driven Page Object structure:

- `tests/data`: external JSON test data.
- `tests/pages`: UI locators and interactions.
- `tests/support`: API setup helpers for deterministic prerequisites.
- `tests/specs`: feature specifications and assertions.
- `reports`: archived execution evidence.

The Playwright configuration executes every feature on Chromium, Firefox, and WebKit. Failed tests retain screenshots, video, and traces. Report metadata contains `Run by: 23127414` and an ISO timestamp.

At least three assertion patterns are used, including URL assertions, visibility, text, count, class, input validity, response status, collection length, and boolean ownership checks.

4. Execution summary

Feature	Cases	Executions	Passed	Failed	Report timestamp (UTC)
FR-01	12	36	12	24	2026-08-14T12:41:07.395Z
FR-11	15	45	42	3	2026-08-14T12:32:37.193Z
FR-14	12	36	18	18	2026-08-14T12:38:54.878Z
Total	39	117	72	45	

There were nine feature–browser runs: three features multiplied by three browsers. All failures listed below reproduced consistently across all three browsers.

5. FR-01 – Account registration

Scope and data

Twelve cases cover successful registration, required fields, whitespace-only names, invalid email partitions, weak passwords, and the eight-character password boundary. Data is stored in `tests/data/registration.json`.

Results

- 12 passed executions.
- 24 failed executions.
- Eight failing cases reproduced on all three browsers.

Genuine defects

1. A password containing a real special character is rejected because the frontend regular expression requires whitespace instead.
2. A whitespace-only name is accepted.
3. Invalid email forms are accepted because the field is `type="text"` and neither frontend nor backend validates the format.

Human review of AI output

The initial generated approach could allow email tests to fail for the wrong reason: the flawed password expression blocked the request before email validation was reached. The final tests use a password containing whitespace only in the targeted invalid-email cases to pass the SUT's flawed gate and isolate the email behavior. Unique email values were added to prevent cross-browser collisions. The JSON loader was also changed from `import.meta` to a working-directory path after the first execution exposed a module-loading error. Form-scoped locators were retained only where the SUT lacks `htmlFor` label associations.

6. FR-11 – User order history

Scope and data

Fifteen cases cover authenticated access, empty state, unauthenticated access, ownership isolation, five translated statuses and colors, required columns, currency formatting, and 0/1 boundaries. Data is stored in `tests/data/order-history.json`.

API setup creates unique users and orders for each worker. Order statuses follow the backend's valid transition path before the UI assertion is made.

Results

- 42 passed executions.
- 3 failed executions.
- The same currency defect reproduced on all three browsers.

Genuine defect

The UI calls `toLocaleString()` without specifying `vi-VN`, producing `1,250,000` ■ in the test environment instead of the required Vietnamese representation `1.250.000` ■.

Human review of AI output

Fixed waits were avoided. The Page Object waits for the real `my-orders` response before checking the table. Data isolation is asserted against every returned `user_id`, not only by checking the visible row count. Boundary cases use isolated users or controlled category-like fixtures rather than relying on an unknown shared database state.

7. FR-14 – Category management

Scope and data

Twelve cases cover creation, list display, deletion, blank and whitespace names, referenced categories, missing IDs, duplicates, non-admin access, one-character names, and 0/1 list boundaries. Data is stored in `tests/data/categories.json`.

Results

- 18 passed executions.
- 18 failed executions.
- Six failing cases reproduced on all three browsers.

Genuine defects

1. Empty and whitespace-only names are accepted.
2. Duplicate names are accepted.
3. A category referenced by a product can be deleted.
4. Deleting a nonexistent ID returns success.
5. A regular user token can create categories because role authorization is missing.

Human review of AI output

The final suite checks HTTP status codes for negative API behavior rather than treating any completed request as success. It creates its own referenced category and product instead of depending on seed IDs. The 0/1 rendering boundaries mock only the category GET response, keeping the UI assertion deterministic without rewriting the SUT database.

8. Defect analysis

Nine distinct bugs are documented in docs/Bug Report .md. A failed automated test is classified as a SUT defect only when:

1. the assertion matches the written requirement;
2. the failure repeats across Chromium, Firefox, and WebKit;
3. screenshot, trace, or response evidence exists; and
4. the automation path was reviewed for selector, wait, data, and assertion errors.

9. AI gap analysis

AI accelerated scaffolding and repetitive conversion, but it initially lacked three kinds of context: the intentional bugs in the SUT, the need for single-fault test isolation, and shared database behavior across browser workers. Human review added unique test data, API prerequisites, status-transition setup, deterministic boundary fixtures, report metadata validation, and database restoration. AI output was therefore treated as a draft, not as accepted test evidence.

10. Reproducibility and evidence

- Reports: reports/fr01, reports/fr11, and reports/fr14.
- Test data: tests/data.
- Test scripts: tests/specs.
- Evidence screenshots: docs/bug-evidence.
- Git history: eight commits that modify .spec.ts files.
- Agent Skill: agent-skills/eshop-playwright-automation.

11. Remaining student-produced evidence

The following cannot be generated or fabricated by AI and must be completed by the student:

- An unlisted YouTube video of at least five minutes with Vietnamese narration.
- Face-cam or terminal output from whoami and hostname.
- A live demonstration of the Agent Skill.
- GitHub Issues created in the student's public repository with screenshots.

Prepared scripts and issue-ready text are included in docs/Video Script.md and docs/GitHub Issue Drafts.md.

12. Conclusion

The completed suite automates 39 test cases and 117 browser executions. It preserves failing assertions that expose nine repeatable defects and includes data files, scripts, multi-browser HTML reports, human-review notes, commit evidence, and a validated reusable Agent Skill.